

SKILL WEEK-8

NAME:CH.SAI VIHAMSH

ROLL:2520090090

SESSION-15

Task1: To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

```
saivihamsh@Varshini-PC:~$ cd ~/OSSP/skill
saivihamsh@Varshini-PC:~/OSSP/skill$ mkdir -p week8/task1
saivihamsh@Varshini-PC:~/OSSP/skill$ cd week8/task1
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ ls -l
total 0
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ nano combined_redirect.c
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ gcc combined_redirect.c
-o T1
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ ls -l
total 24
-rwxr-xr-x 1 madhulika madhulika 16864 Sep 17 16:27 T1
-rw-r--r-- 1 madhulika madhulika 3106 Sep 17 16:27 combined_redirect.c
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ ./T1
=== Skill Week 8 - Task 1 ===
Combined Redirection

Enter command: sh -c 'echo Normal_Output; echo Error_Output >&2' > combined.
txt 2>&1
Output file: &2' > combined.txt
Output file opened successfully.

Both stdout and stderr were redirected.
Child exit status: 2
File descriptors closed successfully.
Combined redirection test completed.
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ cat combined.txt
cat: combined.txt: No such file or directory
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ ./T1
=== Skill Week 8 - Task 1 ===
Combined Redirection

Enter command: sh -c 'echo First; echo Second >&2; echo Third' > order.txt 2
>&1
Output file: &2; echo Third' > order.txt
Output file opened successfully.

Both stdout and stderr were redirected.
Child exit status: 2
File descriptors closed successfully.
Combined redirection test completed.
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_COMMAND 200

int main()
{
    char command[MAX_COMMAND];
    char filename[MAX_COMMAND];
    char *args[20];
    char *token;
    char *redirect;
    int fd;
    int count;
    int status;
    pid_t pid;

    printf("=== Skill Week 8 - Task 1 ===\n");
    printf("Combined Redirection\n\n");

    printf("Enter command: ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
    {
        printf("Failed to read command.\n");
        return 1;
    }

    command[strcspn(command, "\n")] = '\0';

    /*
     * Look for combined redirection:

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

    /*
     * Redirect stderr to the same file
     */
    if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1)
    {
        perror("dup2 stderr");
        close(fd);
        exit(1);
    }

    close(fd);

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

close(fd);

waitpid(pid, &status, 0);

printf("\nBoth stdout and stderr were redirected.\n");

if (WIFEXITED(status))
{
    printf("Child exit status: %d\n",
        WEXITSTATUS(status));
}

printf("File descriptors closed successfully.\n");
printf("Combined redirection test completed.\n");

return 0;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Observation: The program successfully parsed combined redirection and redirected both standard output and standard error to the same file. File descriptors were duplicated using `dup2()`, allowing normal output and error output to be merged into a single stream. The generated files were verified to contain both types of output, and the ordering of the messages was preserved during execution. The program also handled invalid commands and missing output filenames appropriately, demonstrating correct behavior for different edge cases.

Task2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

```
saiviamsh@Varshini-PC:~/OSSP/skill$ mkdir -p week8/task2
saiviamsh@Varshini-PC:~/OSSP/skill$ cd week8/task2
saiviamsh@Varshini-PC:~/OSSP/skill/week8/task2$ nano complex_pipeline.c
saiviamsh@Varshini-PC:~/OSSP/skill/week8/task2$ gcc complex_pipeline.c -
o T2
saiviamsh@Varshini-PC:~/OSSP/skill/week8/task2$ ls -l
total 36
-rwxr-xr-x 1 madhulika madhulika 21128 Sep 17 16:33 T2
-rw-r--r-- 1 madhulika madhulika 8202 Sep 17 16:33 complex_pipeline.c
saiviamsh@Varshini-PC:~/OSSP/skill/week8/task2$ echo "Hello Linux" > inp
ut.txt
saiviamsh@Varshini-PC:~/OSSP/skill/week8/task2$ cat input.txt
Hello Linux
saiviamsh@Varshini-PC:~/OSSP/skill/week8/task2$ ./T2
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections

Enter command: cat input.txt | wc -w

=== Execution Plan ===
Stage 1: cat input.txt
|
Stage 2: wc -w
wc: w: No such file or directory
0
```

```

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
saiviamsh@Varshini-PC:~/OSSP/skill/week8/task2$ ./T2
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections

Enter command: cat result.txt

=== Execution Plan ===
Stage 1: cat result.txt
cat: $'resu\xF0\xDC\xE1\xA4\xFD\x7F': No such file or directory

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
saiviamsh@Varshini-PC:~/OSSP/skill/week8/task2$ ./T2
=== Skill Week 8 - Task 2 ===
Complex Commands, Pipes and Redirections

Enter command: cat input.txt | grep Linux | wc -w

=== Execution Plan ===
Stage 1: cat input.txt
|
Stage 2: grep Linux
|
Stage 3: wc -w
wc: w: No such file or directory
wc: invalid zero-length file name
0

All pipeline processes completed.
Execution order validated.
Resources cleaned successfully.
Complex pipeline test completed.
saiviamsh@Varshini-PC:~/OSSP/skill/week8/task2$

```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 500
#define MAX_COMMANDS 10
#define MAX_ARGS 20

typedef struct
{
    char command[MAX_INPUT];
    char *args[MAX_ARGS];
    int argc;

    char input_file[100];
    char output_file[100];
    char error_file[100];

    int has_input;
    int has_output;
    int has_error;
} Command;

void trim_spaces(char *str)
{
    char *start = str;
    char *end;

    while (*start == ' ' || *start == '\t')
        start++;

    if (start != str)
        memmove(str, start, strlen(start) + 1);
}

```

```
GNU nano 2.7.1 complex_pipeline.c
if (pos != NULL)
{
    *pos = '\\0';

    char *file = pos + 2;

    while (*file == ' ')
        file++;

    if (*file == '\\0')
    {
        printf("Error: Error output file missing.\n");
        return 0;
    }

    sscanf(file, "%99s", cmd->error_file);
    cmd->has_error = 1;
}

/*
 * Parse output redirection
 */
pos = strstr(temp, ">");

if (pos != NULL)
{
    *pos = '\\0';

    char *file = pos + 1;

    while (*file == ' ')
        file++;

    if (*file == '\\0')
    {
        printf("Error: Output file missing.\n");
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
        * Execute command
        */
        execvp(commands[i].args[0],
                commands[i].args);

        perror("execvp");
        exit(1);
    }
}

/*
 * Parent closes all pipe descriptors
 */
for (int i = 0;
     i < command_count - 1;
     i++)
{
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/*
 * Wait for all processes
 */
for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nAll pipeline processes completed.\n");
printf("Execution order validated.\n");
printf("Resources cleaned successfully.\n");
printf("Complex pipeline test completed.\n");

return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Observation: The program successfully parsed complex commands containing multiple pipeline stages and redirection operators. It generated an execution plan showing the order of commands and their input, output, and error redirections before execution. Pipes were created between consecutive processes, while file descriptors were duplicated to implement input, output, and stderr redirection. Multiple processes were launched and synchronized using `waitpid()`. Different test scenarios, including multiple pipelines, input and output redirection, error redirection, missing files, and invalid pipeline syntax, were used to verify the correct execution and handling of complex commands.

SESSION-16

Task1: To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

```
saivihamsh@Varshini-PC:~/OSSP/skill$ mkdir -p week8/task1
saivihamsh@Varshini-PC:~/OSSP/skill$ cd week8/task1
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ nano background_jobs.c
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ gcc background_jobs.c -o
T1
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ ls -l
total 40
-rw-r--r-- 1 madhulika madhulika 60 Sep 17 16:27 "&2' > combined.txt"
-rw-r--r-- 1 madhulika madhulika 52 Sep 17 16:28 "&2; echo Third' > order
.txt"
-rwxr-xr-x 1 madhulika madhulika 16792 Sep 17 16:41 T1
-rw-r--r-- 1 madhulika madhulika 3783 Sep 17 16:40 background_jobs.c
-rw-r--r-- 1 madhulika madhulika 3106 Sep 17 16:27 combined_redirect.c
-rw-r--r-- 1 madhulika madhulika 34 Sep 17 16:28 invalid.txt
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ ./T1
=== Skill Week 8 - Task 1 ===
Background Job Management

Enter command: sleep 5 &

Background job launched successfully.
Job PID: 967
Command: sleep
Parent process continues immediately.

Monitoring background job...
Job PID 967 is still running.
Job PID 967 is still running.

Background job monitoring completed.
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task1$ ./T1
=== Skill Week 8 - Task 1 ===
Background Job Management

Enter command: sleep 1 &

Background job launched successfully.
Job PID: 978
Command: sleep
Parent process continues immediately.
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

```

```

#define MAX_COMMAND 200
#define MAX_ARGS 20

```

```

typedef struct
{
    pid_t pid;
    char command[MAX_COMMAND];
    int running;
} Job;

```

```

Job job;

```

```

void trim_spaces(char *str)
{
    char *start = str;
    char *end;

    while (*start == ' ' || *start == '\t')
        start++;

    if (start != str)
        memmove(str, start, strlen(start) + 1);

    if (strlen(str) == 0)
        return;

    end = str + strlen(str) - 1;

    while (end >= str &&
           (*end == ' ' || *end == '\t'))

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

GNU nano 8.7.1 background_jobs.c *

```

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

/*
 * Parent stores job information
 */
job.pid = pid;
strcpy(job.command, command);
job.running = 1;

printf("\nBackground job launched successfully.\n");
printf("Job PID: %d\n", job.pid);
printf("Command: %s\n", job.command);
printf("Parent process continues immediately.\n");

/*
 * Return prompt immediately.
 * Do not use normal wait().
 */
printf("\nMonitoring background job...\n");

sleep(1);

monitor_job();

sleep(1);

monitor_job();

printf("\nBackground job monitoring completed.\n");

return 0;
}

```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

Observation: The program successfully detected the background operator & and launched the specified command as a child process using fork() and execvp(). The parent process did not use a blocking wait() and therefore continued immediately after launching the background process. The process ID and command information were stored in a job structure, and the execution status was monitored using waitpid() with the WNOHANG option. Different background commands were tested, including commands that were still running, commands that completed quickly, and invalid commands. The program also detected invalid background-job syntax when the & operator or command was missing.

Task2: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

```
saivihamsh@Varshini-PC:~/OSSP/skill$ cd week8/task2
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task2$ nano job_table.c
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task2$ gcc job_table.c -o T2
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task2$ ls -l
total 44
-rwxr-xr-x 1 madhulika madhulika 16880 Sep 17 16:47 T2
-rw-r--r-- 1 madhulika madhulika 8202 Sep 17 16:33 complex_pipeline.c
-rw-r--r-- 1 madhulika madhulika 12 Sep 17 16:33 input.txt
-rw-r--r-- 1 madhulika madhulika 6182 Sep 17 16:47 job_table.c
saivihamsh@Varshini-PC:~/OSSP/skill/week8/task2$ ./T2
=== Skill Week 8 - Task 2 ===
Job Table and Process Group Management

--- MENU ---
1. Add Background Job
2. Display Job Table
3. Update Job Status
4. Remove Completed Jobs
5. Validate Job Records
6. Exit
Enter choice: 1
Enter command name: sleep
Enter duration in seconds: 5

Job added successfully.
Job ID       : 1
PID          : 1071
Process Group: 1071
State        : RUNNING

--- MENU ---
1. Add Background Job
2. Display Job Table
3. Update Job Status
4. Remove Completed Jobs
5. Validate Job Records
6. Exit
```

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 10
#define MAX_COMMAND 100

typedef enum
{
    JOB_RUNNING,
    JOB_COMPLETED,
    JOB_FAILED
} JobState;

typedef struct
{
    int job_id;
    pid_t pid;
    pid_t process_group;
    char command[MAX_COMMAND];
    JobState state;
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

const char *state_name(JobState state)
{
    switch (state)
    {
        case JOB_RUNNING:
            return "RUNNING";

        case JOB_COMPLETED:

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

```

GNU nano 8.7.1      job_table.c *
    }
    else if (choice == 4)
    {
        remove_completed_jobs();
    }
    else if (choice == 5)
    {
        validate_jobs();
    }
    else if (choice == 6)
    {
        /*
         * Clean up any remaining jobs.
         */
        for (int i = 0; i < job_count; i++)
        {
            if (jobs[i].state == JOB_RUNNING)
            {
                kill(jobs[i].pid, SIGTERM);
                waitpid(jobs[i].pid, NULL, 0);
            }
        }

        printf("\nAll resources cleaned successfully.\n");
        printf("Program terminated.\n");

        break;
    }
    else
    {
        printf("Invalid menu choice.\n");
    }
}

return 0;

```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify

Observation: The program successfully created a job table for managing background processes and stored information such as job ID, process ID, process group ID, command name, and execution state. Background jobs were launched as separate processes and placed into their own process groups. The job table was displayed and validated, while the `waitpid()` function with `WNOHANG` was used to monitor process completion without blocking the program. When a job completed, its state was updated and the corresponding job record could be removed from the table. The program also handled invalid durations and cleaned up remaining running processes when exiting.